Final Class Project Ryan Neighbor ANA 500

In [542…
```
hypothesis = "H₁: Biometric measures - specifically age, fasting blood sugar (Fasti
"H₀: Serum cholesterol levels are equally or more predictive of future heart diseas
```

Out[542…
```
'H₀: Serum cholesterol levels are equally or more predictive of future heart disea
se risk compared to biometric measures such as age, FastingBS, MaxHR, and Oldpeak,
when controlling for other relevant health indicators.'
```

In [118…
```
import matplotlib.pyplot as plt
import pandas as pd
import numpy as np
!pip install openpyxl
```

```
Requirement already satisfied: openpyxl in c:\users\nanoo\anaconda3\lib\site-package
s (3.1.5)
Requirement already satisfied: et-xmlfile in c:\users\nanoo\anaconda3\lib\site-packa
ges (from openpyxl) (1.1.0)
```

In [239…
```
# 1a Acquire - identify data sets, retrieve data, query data
# Designate File Path
file_path = r"C:\Users\Nanoo\OneDrive\Desktop\ANA 500\heart disease.xlsx"

# Use read_excel for .xlsx files
df = pd.read_excel(file_path)
```

In [241…
```
# 1b Acquire - identify data sets, retrieve data, query data
# Check First 5 entrys
df.head()
```

Out[241…

	Age	Sex	ChestPainType	RestingBP	Cholesterol	FastingBS	RestingECG	MaxHR	Exer
0	40	M	ATA	140	289	0	Normal	172	
1	49	F	NAP	160	180	0	Normal	156	
2	37	M	ATA	130	283	0	ST	98	
3	48	F	ASY	138	214	0	Normal	108	
4	54	M	NAP	150	195	0	Normal	122	

In [243…
```
# 1c Acquire - identify data sets, retrieve data, query data
# Missing Value Check
df.isnull().sum()
```

Out[243…

```
Age               0
Sex               0
ChestPainType     0
RestingBP         0
Cholesterol       0
FastingBS         0
RestingECG        0
MaxHR             0
ExerciseAngina    0
Oldpeak           0
ST_Slope          0
HeartDisease      0
dtype: int64
```

In [245…

```
# 1d Acquire - identify data sets, retrieve data, query data
# DF Info Check
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 918 entries, 0 to 917
Data columns (total 12 columns):
 #   Column          Non-Null Count  Dtype  
---  ------          --------------  -----  
 0   Age             918 non-null    int64  
 1   Sex             918 non-null    object 
 2   ChestPainType   918 non-null    object 
 3   RestingBP       918 non-null    int64  
 4   Cholesterol     918 non-null    int64  
 5   FastingBS       918 non-null    int64  
 6   RestingECG      918 non-null    object 
 7   MaxHR           918 non-null    int64  
 8   ExerciseAngina  918 non-null    object 
 9   Oldpeak         918 non-null    float64
 10  ST_Slope        918 non-null    object 
 11  HeartDisease    918 non-null    int64  
dtypes: float64(1), int64(6), object(5)
memory usage: 86.2+ KB
```

In [247…

```
# 1e Acquire - identify data sets, retrieve data, query data
# Dissect Categorical Variable Data Types by looping thhrough each Column and Print
categorical_cols = ['Sex', 'ChestPainType', 'RestingECG', 'ExerciseAngina', 'ST_Slo

for col in categorical_cols:
    print(f"\nColumn: {col}")
    print(df[col].value_counts())
```

```
Column: Sex
Sex
M    725
F    193
Name: count, dtype: int64

Column: ChestPainType
ChestPainType
ASY    496
NAP    203
ATA    173
TA      46
Name: count, dtype: int64

Column: RestingECG
RestingECG
Normal    552
LVH       188
ST        178
Name: count, dtype: int64

Column: ExerciseAngina
ExerciseAngina
N    547
Y    371
Name: count, dtype: int64

Column: ST_Slope
ST_Slope
Flat    460
Up      395
Down     63
Name: count, dtype: int64
```

In [249…

```python
# 1e Acquire - identify data sets, retrieve data, query data
# Dissect Categorical Variable Data Types (Autodetect Method)
categorical_cols = df.select_dtypes(include=['object', 'category']).columns

for col in categorical_cols:
    print(f"\nColumn: {col}")
    print(df[col].value_counts())
```

```
Column: Sex
Sex
M    725
F    193
Name: count, dtype: int64

Column: ChestPainType
ChestPainType
ASY    496
NAP    203
ATA    173
TA      46
Name: count, dtype: int64

Column: RestingECG
RestingECG
Normal    552
LVH       188
ST        178
Name: count, dtype: int64

Column: ExerciseAngina
ExerciseAngina
N    547
Y    371
Name: count, dtype: int64

Column: ST_Slope
ST_Slope
Flat    460
Up      395
Down     63
Name: count, dtype: int64
```

In [251…

```
# 1f Acquire - identify data sets, retrieve data, query data

# List View of Columns/Variables for better Understanding
print(df.columns.tolist())
```

```
['Age', 'Sex', 'ChestPainType', 'RestingBP', 'Cholesterol', 'FastingBS', 'RestingEC
G', 'MaxHR', 'ExerciseAngina', 'Oldpeak', 'ST_Slope', 'HeartDisease']
```

In [253…

```
# 2a Prepare - explore and pre-process

# Summary Statistics
df.describe
```

Out[253…

```
<bound method NDFrame.describe of      Age Sex ChestPainType  RestingBP  Cholester
ol  FastingBS RestingECG  \
0     40   M           ATA        140          289          0     Normal
1     49   F           NAP        160          180          0     Normal
2     37   M           ATA        130          283          0         ST
3     48   F           ASY        138          214          0     Normal
4     54   M           NAP        150          195          0     Normal
..   ...  ..           ...        ...          ...        ...        ...
913   45   M            TA        110          264          0     Normal
914   68   M           ASY        144          193          1     Normal
915   57   M           ASY        130          131          0     Normal
916   57   F           ATA        130          236          0        LVH
917   38   M           NAP        138          175          0     Normal

     MaxHR ExerciseAngina  Oldpeak ST_Slope  HeartDisease
0      172              N      0.0       Up             0
1      156              N      1.0     Flat             1
2       98              N      0.0       Up             0
3      108              Y      1.5     Flat             1
4      122              N      0.0       Up             0
..     ...            ...      ...      ...           ...
913    132              N      1.2     Flat             1
914    141              N      3.4     Flat             1
915    115              Y      1.2     Flat             1
916    174              N      0.0     Flat             1
917    173              N      0.0       Up             0

[918 rows x 12 columns]>
```

In [255…

```
# 2b Prepare – explore and pre-process

# Summary Statistics
df[['Age','Cholesterol','MaxHR', 'FastingBS', 'Oldpeak']].describe()
```

Out[255…

	Age	Cholesterol	MaxHR	FastingBS	Oldpeak
count	918.000000	918.000000	918.000000	918.000000	918.000000
mean	53.510893	198.799564	136.809368	0.233115	0.887364
std	9.432617	109.384145	25.460334	0.423046	1.066570
min	28.000000	0.000000	60.000000	0.000000	-2.600000
25%	47.000000	173.250000	120.000000	0.000000	0.000000
50%	54.000000	223.000000	138.000000	0.000000	0.600000
75%	60.000000	267.000000	156.000000	0.000000	1.500000
max	77.000000	603.000000	202.000000	1.000000	6.200000

In [257…

```
# 2c Prepare – explore and pre-process
import seaborn as sns
import matplotlib.pyplot as plt
```

```
#Correlation Heatmap for Continuous Data
plt.figure(figsize=(10,6))
sns.heatmap(df.corr(numeric_only=True), annot=True, cmap='coolwarm')
plt.title("Correlation Heatmap for Heart Disease Data Set")
plt.tight_layout()
plt.show()
```

In [347…

```
# 2d Prepare - explore and pre-process

# Exploratory Filtering
df_filtered = df[(df['Age'] > 50) & (df['Cholesterol'] < 200)]
```

In [349…

```
# 2e Prepare - explore and pre-process
# Summary Statistics
df_filtered[['Age','Cholesterol']].describe()
```

Out[349…

	Age	Cholesterol
count	222.000000	222.000000
mean	59.243243	65.504505
std	5.823523	85.598862
min	51.000000	0.000000
25%	55.000000	0.000000
50%	59.000000	0.000000
75%	63.000000	170.750000
max	77.000000	199.000000

In [351…

```
# Remove rows where cholesterol is zero

df_clean = df[df['Cholesterol'] != 0]
```

In [353…

```
# Summary stats for cleaned cholesterol

print(df_clean['Cholesterol'].describe())
```

```
count    746.000000
mean     244.635389
std       59.153524
min       85.000000
25%      207.250000
50%      237.000000
75%      275.000000
max      603.000000
Name: Cholesterol, dtype: float64
```

In [355…

```
# 2f Prepare - explore and pre-process

# Normalize Continuous Values
# List of numeric columns to normalize
numeric_cols = ['Age', 'Cholesterol', 'MaxHR', 'FastingBS', 'Oldpeak']

# Apply normalization
df_normalized = df_clean[numeric_cols].apply(lambda x: round((x - x.min()) / (x.max

# Added normalized columns back to the original DataFrame
for col in df_normalized.columns:
    df[f'{col}_normalized'] = df_normalized[col]
```

In [357…

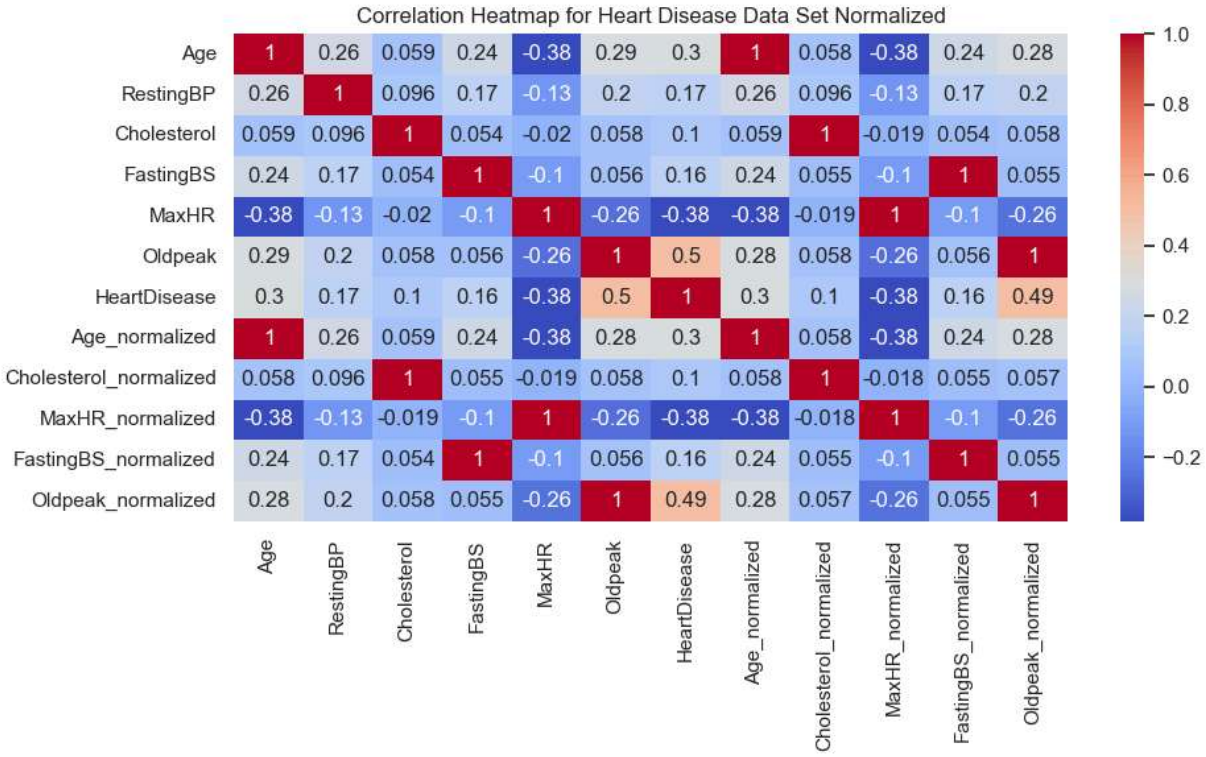
```
# 2e Prepare - explore and pre-process

# Summary Statistics Normalized Dataframe
df_normalized.describe()
```

Out[357…

	Age	Cholesterol	MaxHR	FastingBS	Oldpeak
count	746.000000	746.000000	746.000000	746.000000	746.000000
mean	0.507627	0.308190	0.535094	0.167560	0.159960
std	0.193804	0.114254	0.184083	0.373726	0.168317
min	0.000000	0.000000	0.000000	0.000000	0.000000
25%	0.370000	0.240000	0.400000	0.000000	0.020000
50%	0.530000	0.290000	0.530000	0.000000	0.100000
75%	0.630000	0.370000	0.680000	0.000000	0.250000
max	1.000000	1.000000	1.000000	1.000000	1.000000

In [359…

```
# 2f Prepare - explore and pre-process

#Correlation Heatmap for Normalized Data
plt.figure(figsize=(10,6))
sns.heatmap(df_clean.corr(numeric_only=True), annot=True, cmap='coolwarm')
plt.title("Correlation Heatmap for Heart Disease Data Set Normalized")
plt.tight_layout()
plt.show()
```



When comparing both non & normalized data for correlation: Age vs MaxHR shows promise Heart Disease vs MaxHR shows promise Cholesterol vs fastingBS shows promise Cholesterol vs MaxHR shows promise

In [362…

```
# 2g Prepare - explore and pre-process


# Bin Age into 4 equal groups
df_clean['AgeGroup'] = pd.cut(df['Age'], bins=[0, 25, 50, 75, 100],
                          labels=['0-25', '26-50', '51-75', '76-100'],
                          right=True)

# Group by AgeGroups and summarize Cholesterol

df_clean.groupby('AgeGroup')['Cholesterol'].agg(['mean', 'max', 'min', 'count'])
```

```
C:\Users\Nanoo\AppData\Local\Temp\ipykernel_1928\825285292.py:5: SettingWithCopyWarn
ing:


A value is trying to be set on a copy of a slice from a DataFrame.
Try using .loc[row_indexer,col_indexer] = value instead

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/u
ser_guide/indexing.html#returning-a-view-versus-a-copy

C:\Users\Nanoo\AppData\Local\Temp\ipykernel_1928\825285292.py:11: FutureWarning:

The default of observed=False is deprecated and will be changed to True in a future
version of pandas. Pass observed=False to retain current behavior or observed=True t
o adopt the future default and silence this warning.
```

Out[362…

AgeGroup	mean	max	min	count
0–25	NaN	NaN	NaN	0
26–50	239.099291	529.0	117.0	282
51–75	248.450000	603.0	85.0	460
76–100	196.250000	304.0	113.0	4

In [364…

```
# Correlation matrix for numeric columns only
correlation_table = df_clean.corr(numeric_only=True)

# Display the table
print(correlation_table)
```

	Age	RestingBP	Cholesterol	FastingBS	MaxHR
Age	1.000000	0.259865	0.058758	0.241338	-0.382112
RestingBP	0.259865	1.000000	0.095939	0.173765	-0.125774
Cholesterol	0.058758	0.095939	1.000000	0.054012	-0.019856
FastingBS	0.241338	0.173765	0.054012	1.000000	-0.102710
MaxHR	-0.382112	-0.125774	-0.019856	-0.102710	1.000000
Oldpeak	0.286006	0.198575	0.058488	0.055568	-0.259533
HeartDisease	0.298617	0.173242	0.103866	0.160594	-0.377212
Age_normalized	0.999891	0.259282	0.058620	0.241411	-0.381745
Cholesterol_normalized	0.057729	0.095589	0.999673	0.054893	-0.019159
MaxHR_normalized	-0.381828	-0.125503	-0.018615	-0.102954	0.999887
FastingBS_normalized	0.241338	0.173765	0.054012	1.000000	-0.102710
Oldpeak_normalized	0.284762	0.198391	0.057586	0.054520	-0.256571

	Oldpeak	HeartDisease	Age_normalized
Age	0.286006	0.298617	0.999891
RestingBP	0.198575	0.173242	0.259282
Cholesterol	0.058488	0.103866	0.058620
FastingBS	0.055568	0.160594	0.241411
MaxHR	-0.259533	-0.377212	-0.381745
Oldpeak	1.000000	0.495696	0.284949
HeartDisease	0.495696	1.000000	0.297437
Age_normalized	0.284949	0.297437	1.000000
Cholesterol_normalized	0.057765	0.103287	0.057606
MaxHR_normalized	-0.259723	-0.377171	-0.381448
FastingBS_normalized	0.055568	0.160594	0.241411
Oldpeak_normalized	0.999854	0.494206	0.283682

	Cholesterol_normalized	MaxHR_normalized
Age	0.057729	-0.381828
RestingBP	0.095589	-0.125503
Cholesterol	0.999673	-0.018615
FastingBS	0.054893	-0.102954
MaxHR	-0.019159	0.999887
Oldpeak	0.057765	-0.259723
HeartDisease	0.103287	-0.377171
Age_normalized	0.057606	-0.381448
Cholesterol_normalized	1.000000	-0.017935
MaxHR_normalized	-0.017935	1.000000
FastingBS_normalized	0.054893	-0.102954
Oldpeak_normalized	0.056868	-0.256768

	FastingBS_normalized	Oldpeak_normalized
Age	0.241338	0.284762
RestingBP	0.173765	0.198391
Cholesterol	0.054012	0.057586
FastingBS	1.000000	0.054520
MaxHR	-0.102710	-0.256571
Oldpeak	0.055568	0.999854
HeartDisease	0.160594	0.494206
Age_normalized	0.241411	0.283682
Cholesterol_normalized	0.054893	0.056868
MaxHR_normalized	-0.102954	-0.256768
FastingBS_normalized	1.000000	0.054520
Oldpeak_normalized	0.054520	1.000000

Analyze Data

In [367…
```
import seaborn as sns
```

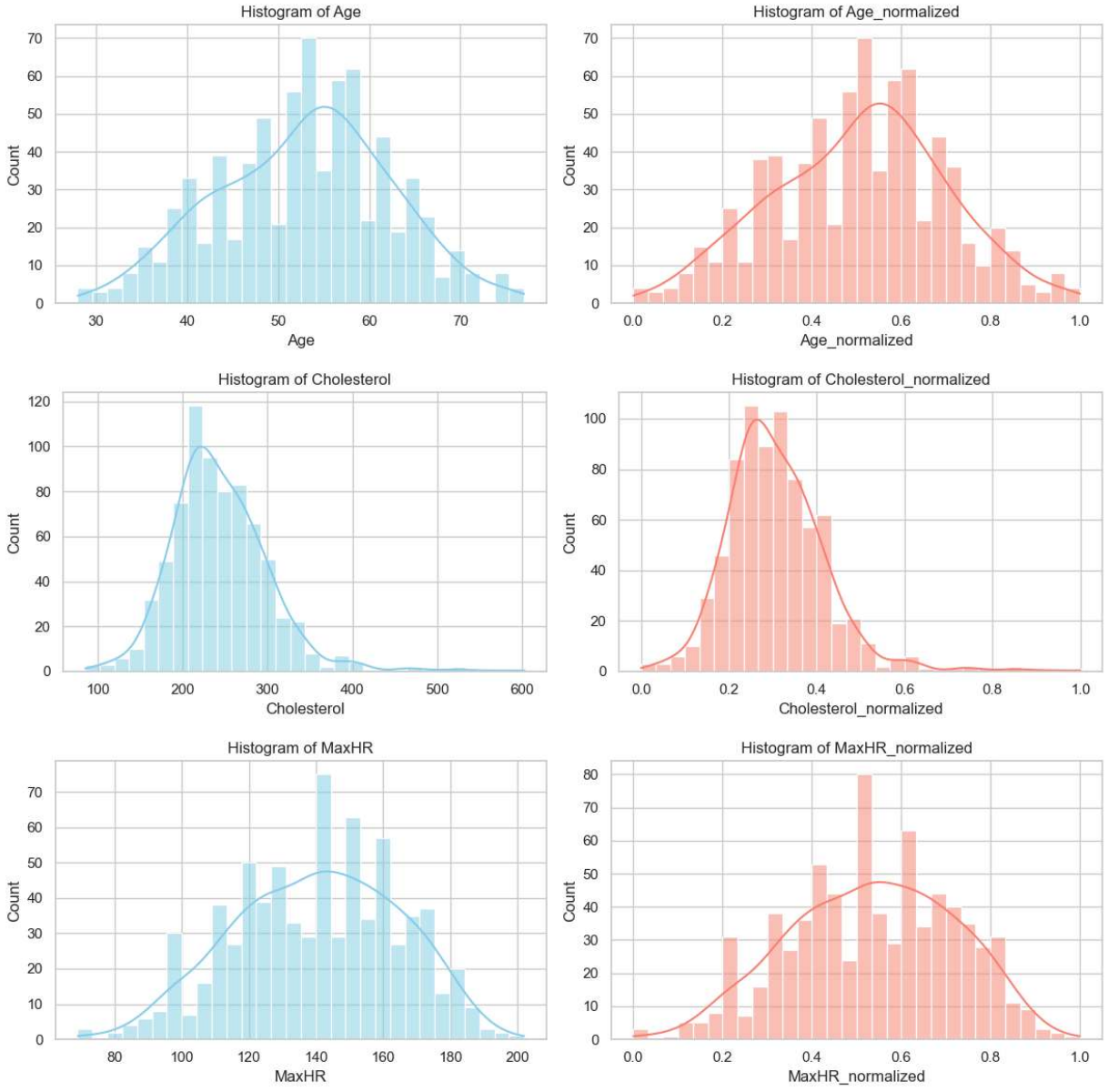
In [369…
```
df_clean.info()
```

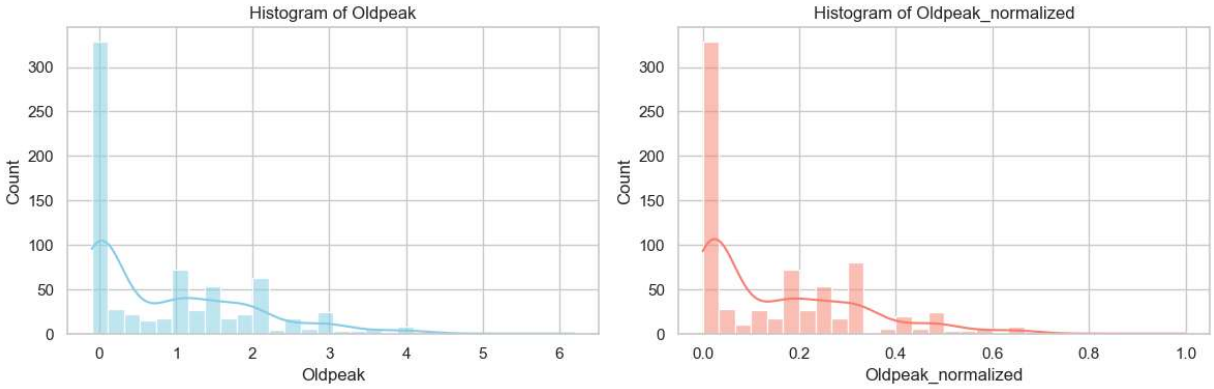
```
<class 'pandas.core.frame.DataFrame'>
Index: 746 entries, 0 to 917
Data columns (total 18 columns):
 #   Column                  Non-Null Count  Dtype
---  ------                  --------------  -----
 0   Age                     746 non-null    int64
 1   Sex                     746 non-null    object
 2   ChestPainType           746 non-null    object
 3   RestingBP               746 non-null    int64
 4   Cholesterol             746 non-null    int64
 5   FastingBS               746 non-null    int64
 6   RestingECG              746 non-null    object
 7   MaxHR                   746 non-null    int64
 8   ExerciseAngina          746 non-null    object
 9   Oldpeak                 746 non-null    float64
 10  ST_Slope                746 non-null    object
 11  HeartDisease            746 non-null    int64
 12  Age_normalized          746 non-null    float64
 13  Cholesterol_normalized  746 non-null    float64
 14  MaxHR_normalized        746 non-null    float64
 15  FastingBS_normalized    746 non-null    float64
 16  Oldpeak_normalized      746 non-null    float64
 17  AgeGroup                746 non-null    category
dtypes: category(1), float64(6), int64(6), object(5)
memory usage: 122.0+ KB
```

In [371…
```
# Matplotlib defaults for readability
plt.rcParams['figure.figsize'] = (8,5)
plt.rcParams['axes.grid'] = True
plt.rcParams['figure.autolayout'] = True
```

In [373…
```
# Set plot style
sns.set(style="whitegrid")

# Variables to plot
variables = ['Age', 'Cholesterol', 'MaxHR', 'Oldpeak']

# Histograms considering original and normalized variables
for var in variables:
    fig, axes = plt.subplots(1, 2, figsize=(12, 4))

    # Original variable
    sns.histplot(df_clean[var], bins=30, kde=True, ax=axes[0], color='skyblue')
    axes[0].set_title(f'Histogram of {var}')
    axes[0].set_xlabel(var)

    # Normalized variable
    norm_var = f"{var}_normalized"
    if norm_var in df_clean.columns:
```

```
        sns.histplot(df_clean[norm_var], bins=30, kde=True, ax=axes[1], color='salm
        axes[1].set_title(f'Histogram of {norm_var}')
        axes[1].set_xlabel(norm_var)
    else:
        axes[1].text(0.5, 0.5, f'{norm_var} not found', ha='center', va='center')
        axes[1].set_title(f'{norm_var} Missing')
        axes[1].axis('off')

    plt.tight_layout()
    plt.show()
```

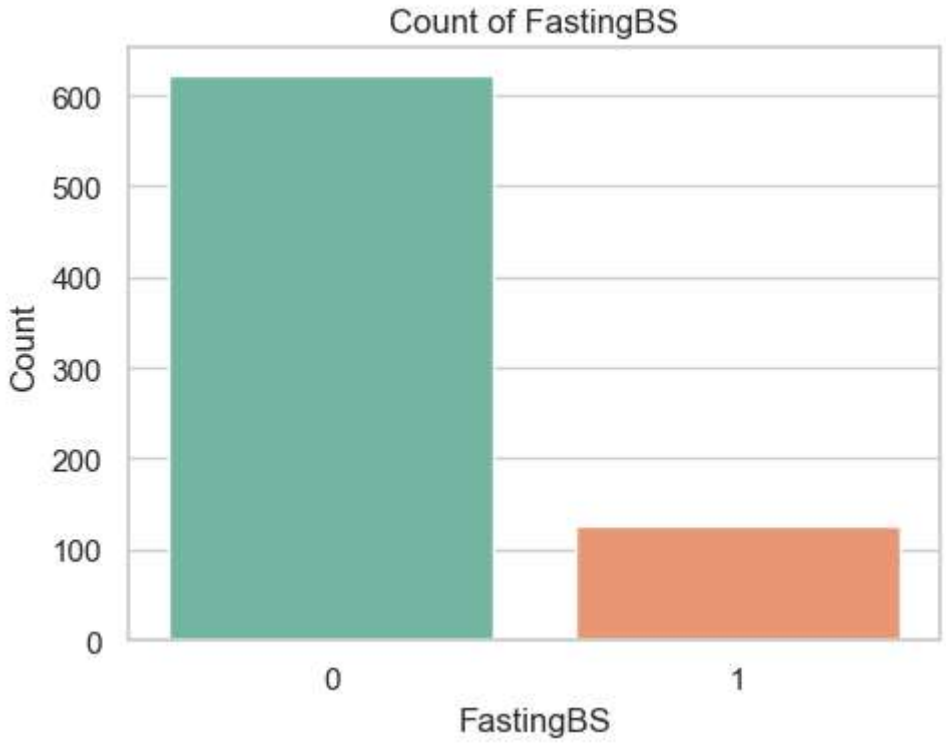




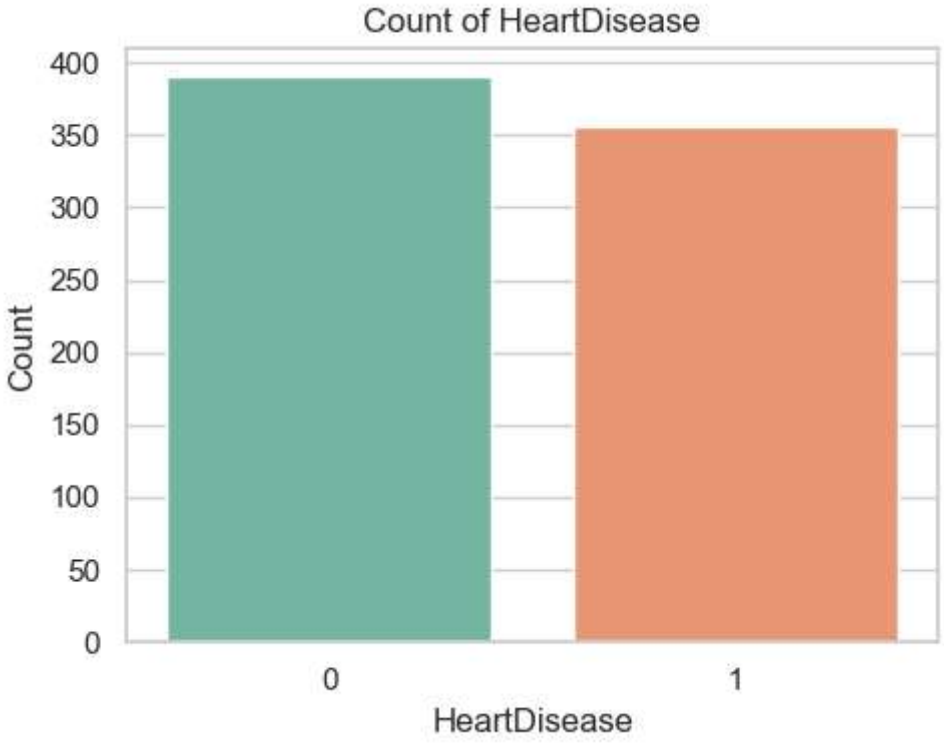
In [375…

```
# Binary categorical variables (bar plot)
binary_vars = ['FastingBS', 'HeartDisease']

for var in binary_vars:
    plt.figure(figsize=(5, 4))
    sns.countplot(x=var, data=df_clean, palette='Set2')
    plt.title(f'Count of {var}')
    plt.xlabel(var)
    plt.ylabel('Count')
    plt.tight_layout()
    plt.show()
```

```
C:\Users\Nanoo\AppData\Local\Temp\ipykernel_1928\4214232699.py:6: FutureWarning:


Passing `palette` without assigning `hue` is deprecated and will be removed in v0.1
4.0. Assign the `x` variable to `hue` and set `legend=False` for the same effect.
```

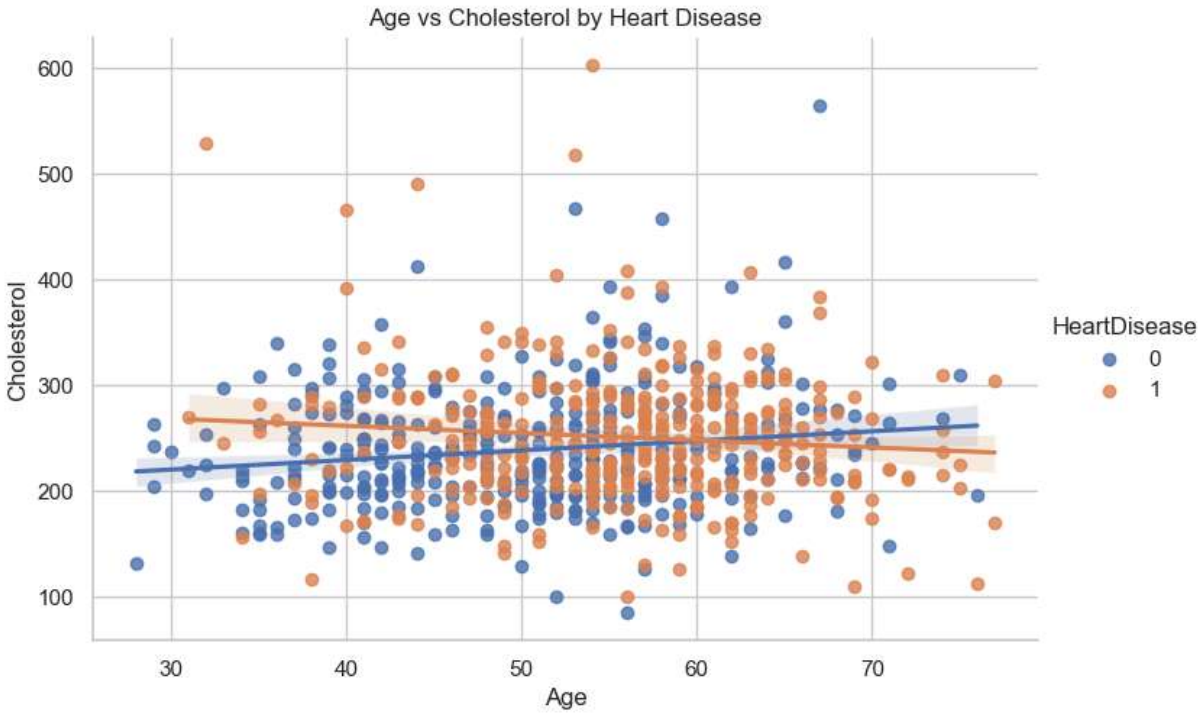


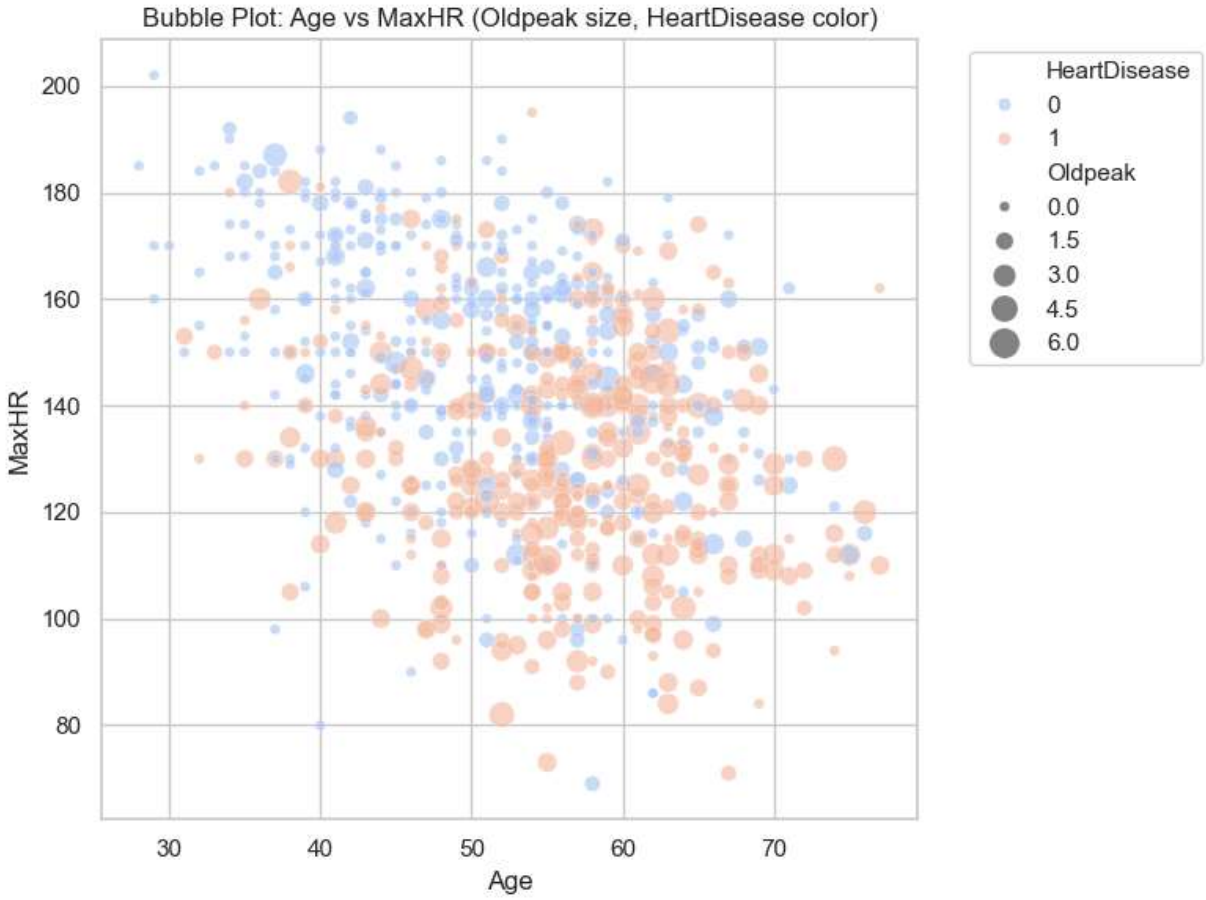
```
C:\Users\Nanoo\AppData\Local\Temp\ipykernel_1928\4214232699.py:6: FutureWarning:


Passing `palette` without assigning `hue` is deprecated and will be removed in v0.1
4.0. Assign the `x` variable to `hue` and set `legend=False` for the same effect.
```



In [377…

```python
# Scatter plot with regression line
sns.lmplot(x='Age', y='Cholesterol', data=df_clean, hue='HeartDisease', aspect=1.5)
plt.title('Age vs Cholesterol by Heart Disease')
plt.show()

# Bubble plot: Age vs MaxHR, bubble size = Oldpeak, color = HeartDisease
plt.figure(figsize=(8, 6))
sns.scatterplot(
    x='Age', y='MaxHR',
    size='Oldpeak', hue='HeartDisease',
    data=df_clean, sizes=(20, 200), alpha=0.6, palette='coolwarm'
)
plt.title('Bubble Plot: Age vs MaxHR (Oldpeak size, HeartDisease color)')
plt.legend(bbox_to_anchor=(1.05, 1), loc=2)
plt.tight_layout()
plt.show()
```





In [379…

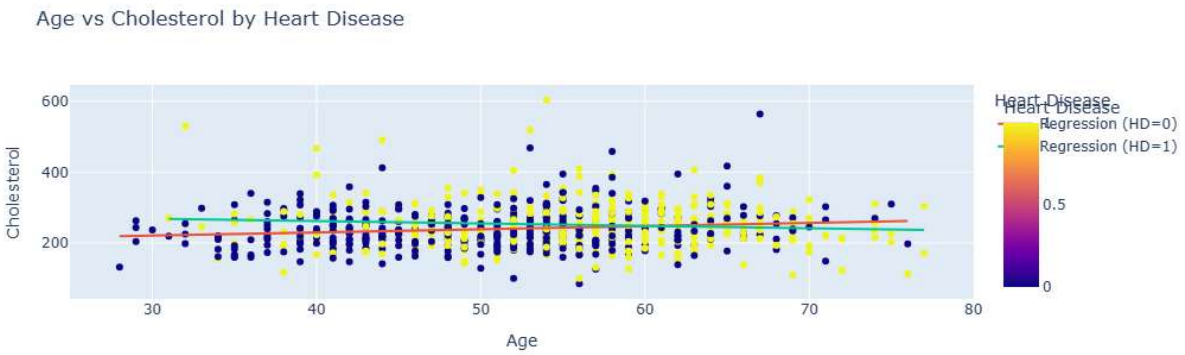
```
!pip install -U plotly
import plotly.express as px
import statsmodels.api as sm
!pip install -U kaleido
```

```
Requirement already satisfied: plotly in c:\users\nanoo\anaconda3\lib\site-packages
(6.3.0)
Requirement already satisfied: narwhals>=1.15.1 in c:\users\nanoo\anaconda3\lib\site
-packages (from plotly) (2.1.2)
Requirement already satisfied: packaging in c:\users\nanoo\anaconda3\lib\site-packag
es (from plotly) (24.1)
Requirement already satisfied: kaleido in c:\users\nanoo\anaconda3\lib\site-packages
(1.0.0)
Requirement already satisfied: choreographer>=1.0.5 in c:\users\nanoo\anaconda3\lib
\site-packages (from kaleido) (1.0.9)
Requirement already satisfied: logistro>=1.0.8 in c:\users\nanoo\anaconda3\lib\site-
packages (from kaleido) (1.1.0)
Requirement already satisfied: orjson>=3.10.15 in c:\users\nanoo\anaconda3\lib\site-
packages (from kaleido) (3.11.2)
Requirement already satisfied: packaging in c:\users\nanoo\anaconda3\lib\site-packag
es (from kaleido) (24.1)
Requirement already satisfied: simplejson>=3.19.3 in c:\users\nanoo\anaconda3\lib\si
te-packages (from choreographer>=1.0.5->kaleido) (3.20.1)
```

In [381…

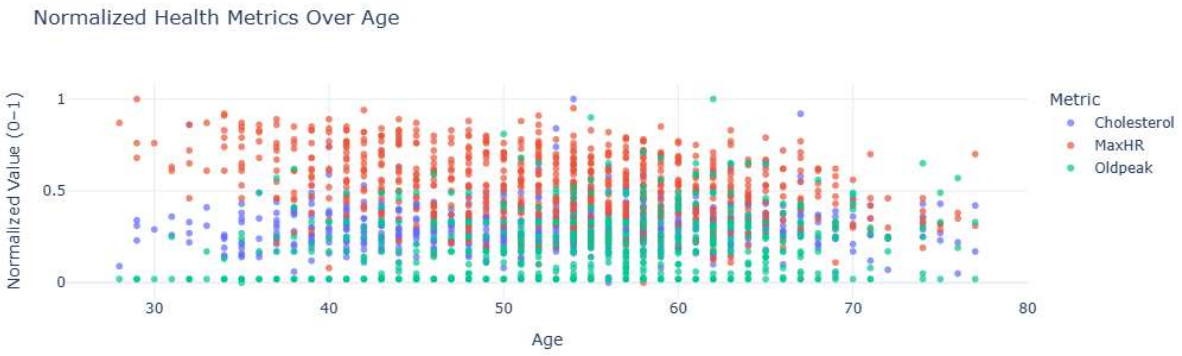
```python
import plotly.express as px
import statsmodels.api as sm


# Fit regression lines separately for each HeartDisease group
def add_regression(df, x, y):
    lines = []
    for label, group in df.groupby('HeartDisease'):
        model = sm.OLS(group[y], sm.add_constant(group[x])).fit()
        pred = model.predict(sm.add_constant(group[x]))
        lines.append(pd.DataFrame({
            x: group[x],
            'Predicted': pred,
            'HeartDisease': label
        }))
    return pd.concat(lines)

# Add regression predictions
df_reg = add_regression(df_clean, 'Age', 'Cholesterol')

# Scatter plot with regression overlay
fig = px.scatter(
    df_clean, x='Age', y='Cholesterol',
    color='HeartDisease',
    title='Age vs Cholesterol by Heart Disease',
    labels={'HeartDisease': 'Heart Disease'},
    hover_data=['Age', 'Cholesterol']
)

# Add regression lines
for label in df_reg['HeartDisease'].unique():
    group = df_reg[df_reg['HeartDisease'] == label]
    fig.add_scatter(x=group['Age'], y=group['Predicted'],
                    mode='lines', name=f'Regression (HD={label})')

fig.update_layout(legend_title_text='Heart Disease')
```

```
fig.write_image("age_vs_cholesterol.png")  # Save for PowerPoint
fig.show()
```



In [494…

```
# Normalized contiuous variables
normalized_vars = [
    'Cholesterol_normalized',
    'MaxHR_normalized',
    'Oldpeak_normalized'
]

# Melt the dataframe to long format
df_long = df_clean[['Age'] + normalized_vars].melt(
    id_vars='Age',
    value_vars=normalized_vars,
    var_name='Metric',
    value_name='Normalized_Value'
)

# Clean up metric names
df_long['Metric'] = df_long['Metric'].str.replace('_normalized', '')

# Create scatter plot
fig = px.scatter(
    df_long,
    x='Age',
    y='Normalized_Value',
    color='Metric',
    title='Normalized Health Metrics Over Age',
    labels={'Normalized_Value': 'Normalized Value (0-1)'},
    opacity=0.7,
    template='plotly_white'
)

fig.write_image("normalized_health_metrics_scatter.png")
fig.show()
```



In [383…
```
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import classification_report, roc_auc_score
```

In [385…
```
# Logistic Regression
# Define features and target
X = df_clean[['Cholesterol_normalized', 'MaxHR_normalized', 'Oldpeak_normalized']]
y = df_clean['HeartDisease']

# Split into train/test
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_sta
```

In [387…
```
# Initialize and train
model = LogisticRegression()
model.fit(X_train, y_train)
```

Out[387…
```
LogisticRegression()
```

In [389…
```
# Predict and evaluate p\Performance
y_pred = model.predict(X_test)
y_prob = model.predict_proba(X_test)[:, 1]

print(classification_report(y_test, y_pred))
print("AUC-ROC:", roc_auc_score(y_test, y_prob))
```

```
              precision    recall  f1-score   support

           0       0.71      0.85      0.77        71
           1       0.83      0.70      0.76        79

    accuracy                           0.77       150
   macro avg       0.77      0.77      0.77       150
weighted avg       0.78      0.77      0.77       150

AUC-ROC: 0.8541629523979318
```

In [391…
```
# Coefficients and odds ratios
coeffs = pd.Series(model.coef_[0], index=X.columns)
odds_ratios = coeffs.apply(lambda x: round(np.exp(x), 2))
```

```
print("Coefficients:\n", coeffs)
print("Odds Ratios:\n", odds_ratios)
```

```
Coefficients:
 Cholesterol_normalized    0.668682
MaxHR_normalized          -3.150142
Oldpeak_normalized         4.687230
dtype: float64
Odds Ratios:
 Cholesterol_normalized      1.95
MaxHR_normalized             0.04
Oldpeak_normalized         108.55
dtype: float64
```

In [393…

```
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.svm import SVC
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import classification_report, roc_auc_score, accuracy_score

# Define target and features
target = 'HeartDisease'

# Normalized features
features_norm = ['Cholesterol_normalized', 'MaxHR_normalized', 'Oldpeak_normalized'

# Raw features
features_raw = ['Cholesterol', 'MaxHR', 'Oldpeak', 'Age']

# Train-test split
X_train_norm, X_test_norm, y_train, y_test = train_test_split(df_clean[features_nor
X_train_raw, X_test_raw, _, _ = train_test_split(df_clean[features_raw], df_clean[t
```

In [395…

```
# Normalized Log Regression Vs. Non Normalized (Normalized)

logreg_norm = LogisticRegression(max_iter=1000)
logreg_norm.fit(X_train_norm, y_train)
y_pred_norm = logreg_norm.predict(X_test_norm)

print("Logistic Regression (Normalized):")
print(classification_report(y_test, y_pred_norm))
print("AUC:", roc_auc_score(y_test, logreg_norm.predict_proba(X_test_norm)[:,1]))
```

```
Logistic Regression (Normalized):
              precision    recall  f1-score   support

           0       0.71      0.87      0.78        71
           1       0.86      0.68      0.76        79

    accuracy                           0.77       150
   macro avg       0.78      0.78      0.77       150
weighted avg       0.79      0.77      0.77       150

AUC: 0.8472098413264396
```

In [397…

```
# Normalized Log Regression Vs. Non Normalized (Non - Normalized i.e. Raw)

scaler = StandardScaler()
X_train_raw_scaled = scaler.fit_transform(X_train_raw)
X_test_raw_scaled = scaler.transform(X_test_raw)

logreg_raw = LogisticRegression(max_iter=1000)
logreg_raw.fit(X_train_raw_scaled, y_train)
y_pred_raw = logreg_raw.predict(X_test_raw_scaled)

print("Logistic Regression (Raw):")
print(classification_report(y_test, y_pred_raw))
print("AUC:", roc_auc_score(y_test, logreg_raw.predict_proba(X_test_raw_scaled)[:,1
```

```
Logistic Regression (Raw):
              precision    recall  f1-score   support

           0       0.72      0.86      0.78        71
           1       0.85      0.70      0.76        79

    accuracy                           0.77       150
   macro avg       0.78      0.78      0.77       150
weighted avg       0.79      0.77      0.77       150

AUC: 0.8563023711891602
```

In [399…

```
# SVM Classification (Normalizded)

svm_model = SVC(kernel='rbf', probability=True)
svm_model.fit(X_train_norm, y_train)
y_pred_svm = svm_model.predict(X_test_norm)

print("SVM (Normalized):")
print(classification_report(y_test, y_pred_svm))
print("AUC:", roc_auc_score(y_test, svm_model.predict_proba(X_test_norm)[:,1]))
```

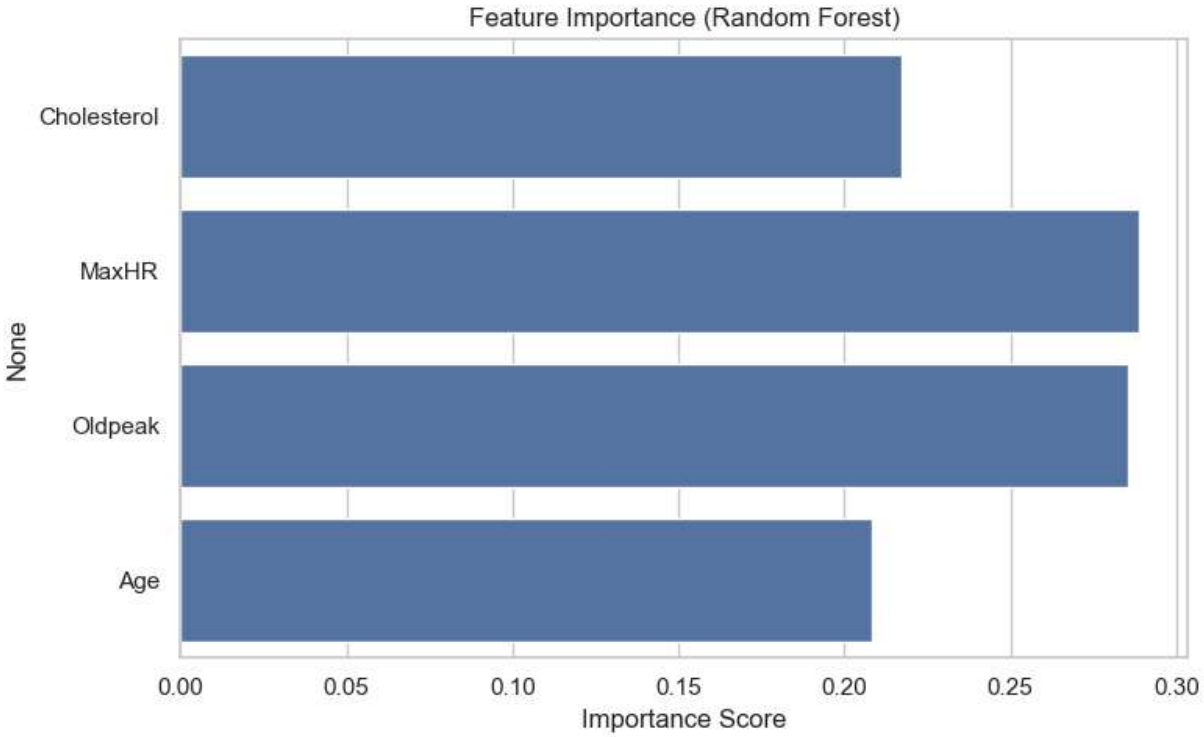
```
SVM (Normalized):
              precision    recall  f1-score   support

           0       0.60      0.69      0.64        71
           1       0.68      0.59      0.64        79

    accuracy                           0.64       150
   macro avg       0.64      0.64      0.64       150
weighted avg       0.65      0.64      0.64       150

AUC: 0.6796220360135495
```

In [401…

```
# Random Forest (Raw)

rf_model = RandomForestClassifier(n_estimators=100, random_state=42)
rf_model.fit(X_train_raw, y_train)
y_pred_rf = rf_model.predict(X_test_raw)

print("Random Forest (Raw):")
print(classification_report(y_test, y_pred_rf))
```

```
print("AUC:", roc_auc_score(y_test, rf_model.predict_proba(X_test_raw)[:,1]))

# Feature importance
import matplotlib.pyplot as plt
import seaborn as sns

feature_importance = pd.Series(rf_model.feature_importances_, index=features_raw)
sns.barplot(x=feature_importance.values, y=feature_importance.index)
plt.title("Feature Importance (Random Forest)")
plt.xlabel("Importance Score")
plt.show()
```

```
Random Forest (Raw):
              precision    recall  f1-score   support

           0       0.68      0.77      0.72        71
           1       0.77      0.67      0.72        79

    accuracy                           0.72       150
   macro avg       0.72      0.72      0.72       150
weighted avg       0.73      0.72      0.72       150

AUC: 0.7900695311107149
```



In [426…

```
%pip install tensorflow
%pip install xgboost
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import GradientBoostingClassifier
from xgboost import XGBClassifier
from sklearn.metrics import (
```

```
    accuracy_score, precision_score, recall_score, f1_score,
    classification_report, confusion_matrix)
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, LSTM
```

```
ERROR: pip's dependency resolver does not currently take into account all the packag
es that are installed. This behaviour is the source of the following dependency conf
licts.
Collecting tensorflow
streamlit 1.37.1 requires protobuf<6,>=3.20, but you have protobuf 6.32.0 which is i
ncompatible.
  Downloading tensorflow-2.20.0-cp312-cp312-win_amd64.whl.metadata (4.6 kB)
```

```
Collecting absl-py>=1.0.0 (from tensorflow)
  Downloading absl_py-2.3.1-py3-none-any.whl.metadata (3.3 kB)
Collecting astunparse>=1.6.0 (from tensorflow)
  Downloading astunparse-1.6.3-py2.py3-none-any.whl.metadata (4.4 kB)
Collecting flatbuffers>=24.3.25 (from tensorflow)Note: you may need to restart the k
ernel to use updated packages.
  Downloading flatbuffers-25.2.10-py2.py3-none-any.whl.metadata (875 bytes)

Collecting gast!=0.5.0,!=0.5.1,!=0.5.2,>=0.2.1 (from tensorflow)
  Downloading gast-0.6.0-py3-none-any.whl.metadata (1.3 kB)
Collecting google_pasta>=0.1.1 (from tensorflow)
  Downloading google_pasta-0.2.0-py3-none-any.whl.metadata (814 bytes)
Collecting libclang>=13.0.0 (from tensorflow)
  Downloading libclang-18.1.1-py2.py3-none-win_amd64.whl.metadata (5.3 kB)
Collecting opt_einsum>=2.3.2 (from tensorflow)
  Downloading opt_einsum-3.4.0-py3-none-any.whl.metadata (6.3 kB)
Requirement already satisfied: packaging in c:\users\nanoo\anaconda3\lib\site-packag
es (from tensorflow) (24.1)
Collecting protobuf>=5.28.0 (from tensorflow)
  Downloading protobuf-6.32.0-cp310-abi3-win_amd64.whl.metadata (593 bytes)
Requirement already satisfied: requests<3,>=2.21.0 in c:\users\nanoo\anaconda3\lib\s
ite-packages (from tensorflow) (2.32.3)
Requirement already satisfied: setuptools in c:\users\nanoo\anaconda3\lib\site-packa
ges (from tensorflow) (75.1.0)
Requirement already satisfied: six>=1.12.0 in c:\users\nanoo\anaconda3\lib\site-pack
ages (from tensorflow) (1.16.0)
Collecting termcolor>=1.1.0 (from tensorflow)
  Downloading termcolor-3.1.0-py3-none-any.whl.metadata (6.4 kB)
Requirement already satisfied: typing_extensions>=3.6.6 in c:\users\nanoo\anaconda3
\lib\site-packages (from tensorflow) (4.11.0)
Requirement already satisfied: wrapt>=1.11.0 in c:\users\nanoo\anaconda3\lib\site-pa
ckages (from tensorflow) (1.14.1)
Collecting grpcio<2.0,>=1.24.3 (from tensorflow)
  Downloading grpcio-1.74.0-cp312-cp312-win_amd64.whl.metadata (4.0 kB)
Collecting tensorboard~=2.20.0 (from tensorflow)
  Downloading tensorboard-2.20.0-py3-none-any.whl.metadata (1.8 kB)
Collecting keras>=3.10.0 (from tensorflow)
  Downloading keras-3.11.3-py3-none-any.whl.metadata (5.9 kB)
Requirement already satisfied: numpy>=1.26.0 in c:\users\nanoo\anaconda3\lib\site-pa
ckages (from tensorflow) (1.26.4)
Requirement already satisfied: h5py>=3.11.0 in c:\users\nanoo\anaconda3\lib\site-pac
kages (from tensorflow) (3.11.0)
Collecting ml_dtypes<1.0.0,>=0.5.1 (from tensorflow)
  Downloading ml_dtypes-0.5.3-cp312-cp312-win_amd64.whl.metadata (9.2 kB)
Requirement already satisfied: wheel<1.0,>=0.23.0 in c:\users\nanoo\anaconda3\lib\si
te-packages (from astunparse>=1.6.0->tensorflow) (0.44.0)
Requirement already satisfied: rich in c:\users\nanoo\anaconda3\lib\site-packages (f
rom keras>=3.10.0->tensorflow) (13.7.1)
Collecting namex (from keras>=3.10.0->tensorflow)
  Downloading namex-0.1.0-py3-none-any.whl.metadata (322 bytes)
Collecting optree (from keras>=3.10.0->tensorflow)
  Downloading optree-0.17.0-cp312-cp312-win_amd64.whl.metadata (34 kB)
Requirement already satisfied: charset-normalizer<4,>=2 in c:\users\nanoo\anaconda3
\lib\site-packages (from requests<3,>=2.21.0->tensorflow) (3.3.2)
Requirement already satisfied: idna<4,>=2.5 in c:\users\nanoo\anaconda3\lib\site-pac
kages (from requests<3,>=2.21.0->tensorflow) (3.7)
```

```
Requirement already satisfied: urllib3<3,>=1.21.1 in c:\users\nanoo\anaconda3\lib\si
te-packages (from requests<3,>=2.21.0->tensorflow) (2.2.3)
Requirement already satisfied: certifi>=2017.4.17 in c:\users\nanoo\anaconda3\lib\si
te-packages (from requests<3,>=2.21.0->tensorflow) (2024.8.30)
Requirement already satisfied: markdown>=2.6.8 in c:\users\nanoo\anaconda3\lib\site-
packages (from tensorboard~=2.20.0->tensorflow) (3.4.1)
Requirement already satisfied: pillow in c:\users\nanoo\anaconda3\lib\site-packages
(from tensorboard~=2.20.0->tensorflow) (10.4.0)
Collecting tensorboard-data-server<0.8.0,>=0.7.0 (from tensorboard~=2.20.0->tensorfl
ow)
  Downloading tensorboard_data_server-0.7.2-py3-none-any.whl.metadata (1.1 kB)
Requirement already satisfied: werkzeug>=1.0.1 in c:\users\nanoo\anaconda3\lib\site-
packages (from tensorboard~=2.20.0->tensorflow) (3.0.3)
Requirement already satisfied: MarkupSafe>=2.1.1 in c:\users\nanoo\anaconda3\lib\sit
e-packages (from werkzeug>=1.0.1->tensorboard~=2.20.0->tensorflow) (2.1.3)
Requirement already satisfied: markdown-it-py>=2.2.0 in c:\users\nanoo\anaconda3\lib
\site-packages (from rich->keras>=3.10.0->tensorflow) (2.2.0)
Requirement already satisfied: pygments<3.0.0,>=2.13.0 in c:\users\nanoo\anaconda3\l
ib\site-packages (from rich->keras>=3.10.0->tensorflow) (2.15.1)
Requirement already satisfied: mdurl~=0.1 in c:\users\nanoo\anaconda3\lib\site-packa
ges (from markdown-it-py>=2.2.0->rich->keras>=3.10.0->tensorflow) (0.1.0)
Downloading tensorflow-2.20.0-cp312-cp312-win_amd64.whl (331.9 MB)
   ---------------------------------------- 0.0/331.9 MB ? eta -:--:--
   ---------------------------------------- 2.1/331.9 MB 14.7 MB/s eta 0:00:23
    --------------------------------------- 7.1/331.9 MB 19.0 MB/s eta 0:00:18
   - -------------------------------------- 11.5/331.9 MB 20.6 MB/s eta 0:00:16
   - -------------------------------------- 15.7/331.9 MB 20.6 MB/s eta 0:00:16
   -- ------------------------------------- 20.2/331.9 MB 20.9 MB/s eta 0:00:15
   -- ------------------------------------- 23.6/331.9 MB 20.2 MB/s eta 0:00:16
   --- ------------------------------------ 27.3/331.9 MB 19.6 MB/s eta 0:00:16
   --- ------------------------------------ 31.5/331.9 MB 19.8 MB/s eta 0:00:16
   ---- ----------------------------------- 35.1/331.9 MB 19.4 MB/s eta 0:00:16
   ---- ----------------------------------- 40.4/331.9 MB 19.9 MB/s eta 0:00:15
   ----- ---------------------------------- 44.8/331.9 MB 20.1 MB/s eta 0:00:15
   ----- ---------------------------------- 49.5/331.9 MB 20.3 MB/s eta 0:00:14
   ------ --------------------------------- 53.2/331.9 MB 20.2 MB/s eta 0:00:14
   ------ --------------------------------- 57.4/331.9 MB 20.0 MB/s eta 0:00:14
   ------- -------------------------------- 61.1/331.9 MB 20.0 MB/s eta 0:00:14
   ------- -------------------------------- 64.7/331.9 MB 19.9 MB/s eta 0:00:14
   -------- ------------------------------- 68.9/331.9 MB 19.7 MB/s eta 0:00:14
   -------- ------------------------------- 72.1/331.9 MB 19.6 MB/s eta 0:00:14
   --------- ------------------------------ 76.0/331.9 MB 19.4 MB/s eta 0:00:14
   --------- ------------------------------ 80.0/331.9 MB 19.4 MB/s eta 0:00:13
   ---------- ----------------------------- 83.9/331.9 MB 19.4 MB/s eta 0:00:13
   ---------- ----------------------------- 88.1/331.9 MB 19.4 MB/s eta 0:00:13
   ---------- ----------------------------- 91.2/331.9 MB 19.3 MB/s eta 0:00:13
   ----------- ---------------------------- 95.7/331.9 MB 19.3 MB/s eta 0:00:13
   ----------- ---------------------------- 100.7/331.9 MB 19.5 MB/s eta 0:00:12
   ------------ --------------------------- 105.1/331.9 MB 19.5 MB/s eta 0:00:12
   ------------ --------------------------- 109.3/331.9 MB 19.6 MB/s eta 0:00:12
   ------------- -------------------------- 114.3/331.9 MB 19.7 MB/s eta 0:00:12
   ------------- -------------------------- 118.5/331.9 MB 19.7 MB/s eta 0:00:11
   -------------- ------------------------- 121.1/331.9 MB 19.5 MB/s eta 0:00:11
   -------------- ------------------------- 124.3/331.9 MB 19.4 MB/s eta 0:00:11
   --------------- ------------------------ 127.9/331.9 MB 19.3 MB/s eta 0:00:11
   --------------- ------------------------ 131.1/331.9 MB 19.3 MB/s eta 0:00:11
```

```
   -------------- ---------------------- 134.0/331.9 MB 19.1 MB/s eta 0:00:11
   --------------- ---------------------- 137.1/331.9 MB 19.0 MB/s eta 0:00:11
   --------------- ---------------------- 140.5/331.9 MB 18.9 MB/s eta 0:00:11
   ---------------- --------------------- 143.4/331.9 MB 18.7 MB/s eta 0:00:11
   ---------------- --------------------- 147.1/331.9 MB 18.7 MB/s eta 0:00:10
   ----------------- -------------------- 148.9/331.9 MB 18.6 MB/s eta 0:00:10
   ----------------- -------------------- 152.3/331.9 MB 18.4 MB/s eta 0:00:10
   ------------------ ------------------- 155.7/331.9 MB 18.4 MB/s eta 0:00:10
   ------------------ ------------------- 159.4/331.9 MB 18.3 MB/s eta 0:00:10
   ------------------- ------------------ 162.0/331.9 MB 18.4 MB/s eta 0:00:10
   ------------------- ------------------ 168.3/331.9 MB 18.4 MB/s eta 0:00:09
   -------------------- ----------------- 172.8/331.9 MB 18.5 MB/s eta 0:00:09
   -------------------- ----------------- 177.2/331.9 MB 18.6 MB/s eta 0:00:09
   --------------------- ---------------- 181.1/331.9 MB 18.7 MB/s eta 0:00:09
   --------------------- ---------------- 184.8/331.9 MB 18.5 MB/s eta 0:00:08
   ---------------------- --------------- 189.0/331.9 MB 18.6 MB/s eta 0:00:08
   ---------------------- --------------- 192.9/331.9 MB 18.6 MB/s eta 0:00:08
   ----------------------- -------------- 197.9/331.9 MB 18.7 MB/s eta 0:00:08
   ----------------------- -------------- 202.1/331.9 MB 18.7 MB/s eta 0:00:07
   ------------------------ ------------- 206.8/331.9 MB 18.8 MB/s eta 0:00:07
   ------------------------ ------------- 211.6/331.9 MB 18.9 MB/s eta 0:00:07
   ------------------------- ------------ 216.3/331.9 MB 18.9 MB/s eta 0:00:07
   ------------------------- ------------ 219.9/331.9 MB 19.0 MB/s eta 0:00:06
   -------------------------- ----------- 223.6/331.9 MB 18.9 MB/s eta 0:00:06
   -------------------------- ----------- 227.5/331.9 MB 18.9 MB/s eta 0:00:06
   --------------------------- ---------- 231.2/331.9 MB 18.9 MB/s eta 0:00:06
   --------------------------- ---------- 235.1/331.9 MB 18.9 MB/s eta 0:00:06
   ---------------------------- --------- 239.6/331.9 MB 18.9 MB/s eta 0:00:05
   ---------------------------- --------- 243.8/331.9 MB 18.9 MB/s eta 0:00:05
   ----------------------------- -------- 247.7/331.9 MB 18.9 MB/s eta 0:00:05
   ----------------------------- -------- 251.7/331.9 MB 19.0 MB/s eta 0:00:05
   ------------------------------ ------- 255.9/331.9 MB 19.0 MB/s eta 0:00:05
   ------------------------------ ------- 260.0/331.9 MB 19.0 MB/s eta 0:00:04
   ------------------------------- ------ 264.5/331.9 MB 19.0 MB/s eta 0:00:04
   ------------------------------- ------ 268.4/331.9 MB 19.0 MB/s eta 0:00:04
   -------------------------------- ----- 272.9/331.9 MB 19.0 MB/s eta 0:00:04
   -------------------------------- ----- 277.1/331.9 MB 19.0 MB/s eta 0:00:03
   --------------------------------- ---- 281.5/331.9 MB 19.0 MB/s eta 0:00:03
   --------------------------------- ---- 285.7/331.9 MB 19.0 MB/s eta 0:00:03
   ---------------------------------- --- 290.5/331.9 MB 19.1 MB/s eta 0:00:03
   ---------------------------------- --- 293.3/331.9 MB 19.0 MB/s eta 0:00:03
   ----------------------------------- -- 297.3/331.9 MB 19.0 MB/s eta 0:00:02
   ----------------------------------- -- 301.5/331.9 MB 19.0 MB/s eta 0:00:02
   ------------------------------------ - 306.2/331.9 MB 19.0 MB/s eta 0:00:02
   ------------------------------------ - 309.9/331.9 MB 19.0 MB/s eta 0:00:02
   ------------------------------------- 314.6/331.9 MB 19.0 MB/s eta 0:00:01
   ------------------------------------- 318.8/331.9 MB 19.0 MB/s eta 0:00:01
   -------------------------------------- 323.5/331.9 MB 19.1 MB/s eta 0:00:01
   -------------------------------------- 327.9/331.9 MB 19.2 MB/s eta 0:00:01
   --------------------------------------- 331.9/331.9 MB 19.1 MB/s eta 0:00:01
   --------------------------------------- 331.9/331.9 MB 19.1 MB/s eta 0:00:01
   --------------------------------------- 331.9/331.9 MB 19.1 MB/s eta 0:00:01
   --------------------------------------- 331.9/331.9 MB 19.1 MB/s eta 0:00:01
   --------------------------------------- 331.9/331.9 MB 19.1 MB/s eta 0:00:01
   ---------------------------------------- 331.9/331.9 MB 17.8 MB/s eta 0:00:00
Downloading absl_py-2.3.1-py3-none-any.whl (135 kB)
```

```
Downloading astunparse-1.6.3-py2.py3-none-any.whl (12 kB)
Downloading flatbuffers-25.2.10-py2.py3-none-any.whl (30 kB)
Downloading gast-0.6.0-py3-none-any.whl (21 kB)
Downloading google_pasta-0.2.0-py3-none-any.whl (57 kB)
Downloading grpcio-1.74.0-cp312-cp312-win_amd64.whl (4.5 MB)
   ---------------------------------------- 0.0/4.5 MB ? eta -:--:--
   ---------------------------------------  4.5/4.5 MB 22.3 MB/s eta 0:00:01
   ---------------------------------------- 4.5/4.5 MB 20.8 MB/s eta 0:00:00
Downloading keras-3.11.3-py3-none-any.whl (1.4 MB)
   ---------------------------------------- 0.0/1.4 MB ? eta -:--:--
   ---------------------------------------- 1.4/1.4 MB 24.4 MB/s eta 0:00:00
Downloading libclang-18.1.1-py2.py3-none-win_amd64.whl (26.4 MB)
   ---------------------------------------- 0.0/26.4 MB ? eta -:--:--
   ----- ---------------------------------- 3.9/26.4 MB 19.6 MB/s eta 0:00:02
   ---------- ----------------------------- 7.9/26.4 MB 19.5 MB/s eta 0:00:01
   ----------------- ---------------------- 12.1/26.4 MB 19.9 MB/s eta 0:00:01
   ------------------------ --------------- 16.8/26.4 MB 19.9 MB/s eta 0:00:01
   ------------------------------ -------- 20.7/26.4 MB 19.8 MB/s eta 0:00:01
   ---------------------------------- ----- 24.6/26.4 MB 19.8 MB/s eta 0:00:01
   ---------------------------------------- 26.4/26.4 MB 18.2 MB/s eta 0:00:00
Downloading ml_dtypes-0.5.3-cp312-cp312-win_amd64.whl (208 kB)
Downloading opt_einsum-3.4.0-py3-none-any.whl (71 kB)
Downloading protobuf-6.32.0-cp310-abi3-win_amd64.whl (435 kB)
Downloading tensorboard-2.20.0-py3-none-any.whl (5.5 MB)
   ---------------------------------------- 0.0/5.5 MB ? eta -:--:--
   ---------------------------------- ----- 4.7/5.5 MB 23.8 MB/s eta 0:00:01
   ---------------------------------------- 5.5/5.5 MB 21.1 MB/s eta 0:00:00
Downloading termcolor-3.1.0-py3-none-any.whl (7.7 kB)
Downloading tensorboard_data_server-0.7.2-py3-none-any.whl (2.4 kB)
Downloading namex-0.1.0-py3-none-any.whl (5.9 kB)
Downloading optree-0.17.0-cp312-cp312-win_amd64.whl (314 kB)
Installing collected packages: namex, libclang, flatbuffers, termcolor, tensorboard-
data-server, protobuf, optree, opt_einsum, ml_dtypes, grpcio, google_pasta, gast, as
tunparse, absl-py, tensorboard, keras, tensorflow
  Attempting uninstall: protobuf
    Found existing installation: protobuf 4.25.3
    Uninstalling protobuf-4.25.3:
      Successfully uninstalled protobuf-4.25.3
Successfully installed absl-py-2.3.1 astunparse-1.6.3 flatbuffers-25.2.10 gast-0.6.0
google_pasta-0.2.0 grpcio-1.74.0 keras-3.11.3 libclang-18.1.1 ml_dtypes-0.5.3 namex-
0.1.0 opt_einsum-3.4.0 optree-0.17.0 protobuf-6.32.0 tensorboard-2.20.0 tensorboard-
data-server-0.7.2 tensorflow-2.20.0 termcolor-3.1.0
Requirement already satisfied: xgboost in c:\users\nanoo\anaconda3\lib\site-packages
(3.0.4)
Requirement already satisfied: numpy in c:\users\nanoo\anaconda3\lib\site-packages
(from xgboost) (1.26.4)
Requirement already satisfied: scipy in c:\users\nanoo\anaconda3\lib\site-packages
(from xgboost) (1.13.1)
Note: you may need to restart the kernel to use updated packages.
```

In [470…

```
# Prepare Data for LR-Hyperparameter Tuning, Gradient Boosting, XG Boost, FF Nueral
X = df_clean[['Cholesterol_normalized','MaxHR_normalized','Oldpeak_normalized','Age
y = df_clean['HeartDisease'].values

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_sta
```

```
# Scale for Models that Require It
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled  = scaler.transform(X_test)
```

In [472…
```
# Logistic Regression + Hyperparameter Tuning Model
lr = LogisticRegression(max_iter=1000)
param_grid = {
    'C': [0.01, 0.1, 1, 10],
    'penalty': ['l1','l2'],
    'solver': ['liblinear']}
grid_lr = GridSearchCV(lr, param_grid, cv=5, scoring='f1')
grid_lr.fit(X_train_scaled, y_train)
best_lr = grid_lr.best_estimator_
y_pred_lr = best_lr.predict(X_test_scaled)
```

In [474…
```
# Gradient Boosting (Raw features) Model
gb = GradientBoostingClassifier(
    n_estimators=100, learning_rate=0.1,
    max_depth=3, random_state=42)
gb.fit(X_train, y_train)
y_pred_gb = gb.predict(X_test)
```

In [476…
```
# XGBoost (Raw features) Model
xgb = XGBClassifier(
    use_label_encoder=False, eval_metric='logloss',
    n_estimators=100, learning_rate=0.1, random_state=42)
xgb.fit(X_train, y_train)
y_pred_xgb = xgb.predict(X_test)
```

```
C:\Users\Nanoo\anaconda3\Lib\site-packages\xgboost\training.py:183: UserWarning:

[11:13:37] WARNING: C:\actions-runner\_work\xgboost\xgboost\src\learner.cc:738:
Parameters: { "use_label_encoder" } are not used.
```

In [478…
```
# Feed-forward Neural Network (NN w/ Scaled Features)
nn = Sequential([
    Dense(16, activation='relu', input_shape=(X_train_scaled.shape[1],)),
    Dense(8, activation='relu'),
    Dense(1, activation='sigmoid')])
nn.compile(optimizer='adam', loss='binary_crossentropy', metrics=['accuracy'])
nn.fit(X_train_scaled, y_train, epochs=10, batch_size=10, verbose=0)
y_pred_nn_prob = nn.predict(X_test_scaled).ravel()
y_pred_nn = (y_pred_nn_prob > 0.5).astype(int)
```

```
C:\Users\Nanoo\anaconda3\Lib\site-packages\keras\src\layers\core\dense.py:92: UserWa
rning:

Do not pass an `input_shape`/`input_dim` argument to a layer. When using Sequential
models, prefer using an `Input(shape)` object as the first layer in the model instea
d.
```

```
5/5 ━━━━━━━━━━━━━━━━━━━━ 0s 22ms/ste ━━━━━━━━━━━━━━━━━━━━ 0s 7ms/step
```

In [460…

```
# LSTM (Treat each row as a 1-step sequence)
X_train_lstm = X_train_scaled.reshape(-1, 1, X_train_scaled.shape[1])
X_test_lstm  = X_test_scaled.reshape(-1, 1, X_test_scaled.shape[1])

lstm = Sequential([
    LSTM(16, input_shape=(1, X_train_scaled.shape[1])),
    Dense(1, activation='sigmoid')
])
lstm.compile(optimizer='adam', loss='binary_crossentropy', metrics=['accuracy'])
lstm.fit(X_train_lstm, y_train, epochs=10, batch_size=10, verbose=0)

y_pred_lstm_prob = lstm.predict(X_test_lstm).ravel()
y_pred_lstm = (y_pred_lstm_prob > 0.5).astype(int)


# 8) LSTM confusion matrix & classification report
print("LSTM Confusion Matrix:")
print(confusion_matrix(y_test, y_pred_lstm))
print("\nLSTM Classification Report:")
print(classification_report(y_test, y_pred_lstm))
```

```
C:\Users\Nanoo\anaconda3\Lib\site-packages\keras\src\layers\rnn\rnn.py:199: UserWarn
ing:

Do not pass an `input_shape`/`input_dim` argument to a layer. When using Sequential
models, prefer using an `Input(shape)` object as the first layer in the model instea
d.
```

```
5/5 ━━━━━━━━━━━━━━━━━━━━ 0s 63ms/ste ━━━━━━━━━━━━━━━━━━━━ 0s 16ms/ste ━━━━━━━━━━━━
━━━━ 0s 18ms/step
LSTM Confusion Matrix:
[[66 12]
 [25 47]]

LSTM Classification Report:
              precision    recall  f1-score   support

           0       0.73      0.85      0.78        78
           1       0.80      0.65      0.72        72

    accuracy                           0.75       150
   macro avg       0.76      0.75      0.75       150
weighted avg       0.76      0.75      0.75       150
```

In [462…

```
# 9) Compare all models
models = {
    'LogisticRegression': y_pred_lr,
    'GradientBoosting':   y_pred_gb,
    'XGBoost':            y_pred_xgb,
    'NeuralNetwork':      y_pred_nn,
    'LSTM':               y_pred_lstm}
rows = []
for name, y_pred in models.items():
    rows.append({
```

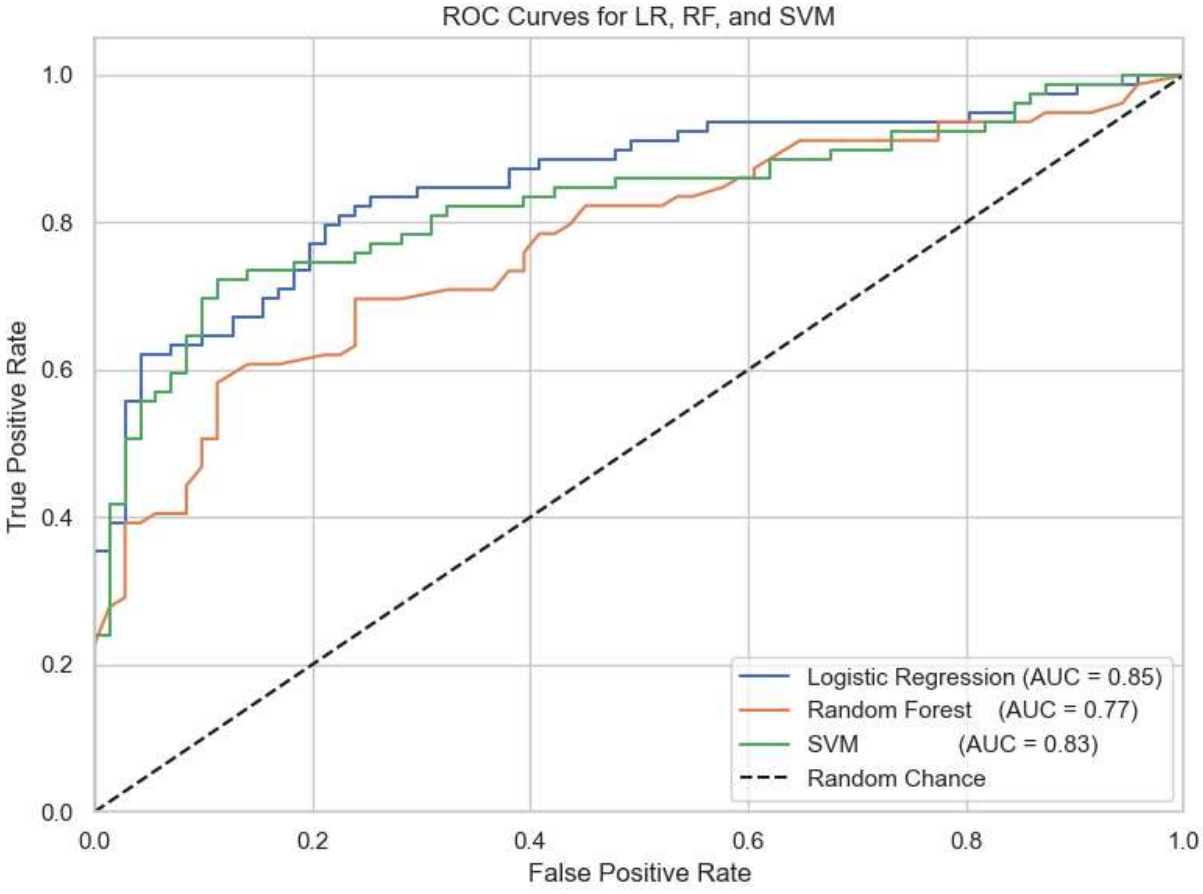
```
        'Model':     name,
        'Accuracy':  accuracy_score(y_test, y_pred),
        'Precision': precision_score(y_test, y_pred),
        'Recall':    recall_score(y_test, y_pred),
        'F1 Score':  f1_score(y_test, y_pred)})

results_df = pd.DataFrame(rows).set_index('Model')
print("\nModel Comparison:\n", results_df)
```

```
Model Comparison:
                     Accuracy  Precision    Recall  F1 Score
Model
LogisticRegression  0.760000   0.810345  0.652778  0.723077
GradientBoosting    0.773333   0.851852  0.638889  0.730159
XGBoost             0.773333   0.827586  0.666667  0.738462
NeuralNetwork       0.740000   0.779661  0.638889  0.702290
LSTM                0.753333   0.796610  0.652778  0.717557
```

In [519…

```
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.linear_model    import LogisticRegression
from sklearn.ensemble        import RandomForestClassifier
from sklearn.svm             import SVC
from sklearn.metrics         import roc_curve, auc

# Logisitc Regression, Random Forest, SVM for ROC Curve
# Redo Define features & target
features_norm = ['Cholesterol_normalized', 'MaxHR_normalized', 'Oldpeak_normalized'
features_raw  = ['Cholesterol', 'MaxHR', 'Oldpeak']
target        = 'HeartDisease'

X_norm = df_clean[features_norm]
X_raw  = df_clean[features_raw]
y      = df_clean[target]

# Split once (same random_state keeps train/test aligned)
X_train_norm, X_test_norm, y_train, y_test = train_test_split(
    X_norm, y, test_size=0.2, random_state=42
)
X_train_raw, X_test_raw, _, _ = train_test_split(
    X_raw, y, test_size=0.2, random_state=42)

# Fit models & get probability scores
# Logistic Regression on normalized features
logreg = LogisticRegression(max_iter=1000)
logreg.fit(X_train_norm, y_train)
y_prob_log = logreg.predict_proba(X_test_norm)[:, 1]
fpr_log, tpr_log, _ = roc_curve(y_test, y_prob_log)
auc_log = auc(fpr_log, tpr_log)

# Random Forest on raw features
rf = RandomForestClassifier(n_estimators=100, random_state=42)
rf.fit(X_train_raw, y_train)
y_prob_rf = rf.predict_proba(X_test_raw)[:, 1]
fpr_rf, tpr_rf, _ = roc_curve(y_test, y_prob_rf)
auc_rf = auc(fpr_rf, tpr_rf)
```

```
# SVM on normalized features
svm = SVC(kernel='rbf', probability=True, random_state=42)
svm.fit(X_train_norm, y_train)
y_prob_svm = svm.predict_proba(X_test_norm)[:, 1]
fpr_svm, tpr_svm, _ = roc_curve(y_test, y_prob_svm)
auc_svm = auc(fpr_svm, tpr_svm)

# Plot all ROC curves
plt.figure(figsize=(8, 6))
plt.plot(fpr_log, tpr_log, label=f'Logistic Regression (AUC = {auc_log:.2f})')
plt.plot(fpr_rf,  tpr_rf,  label=f'Random Forest    (AUC = {auc_rf:.2f})')
plt.plot(fpr_svm, tpr_svm, label=f'SVM              (AUC = {auc_svm:.2f})')

# Chance line
plt.plot([0, 1], [0, 1], 'k--', label='Random Chance')

plt.xlim(0, 1); plt.ylim(0, 1.05)
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.title('ROC Curves for LR, RF, and SVM')
plt.legend(loc='lower right')
plt.grid(True)
plt.tight_layout()
plt.show()
```



In [509…

```
# Rerun this cell First to import everything and silence XGBoost's warning
import warnings
```

```
warnings.filterwarnings("ignore", category=UserWarning, module="xgboost")

import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.ensemble      import GradientBoostingClassifier
from xgboost               import XGBClassifier
from sklearn.neural_network import MLPClassifier
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import LSTM, Dense
from sklearn.preprocessing  import StandardScaler
from sklearn.metrics        import roc_curve, auc
from sklearn.model_selection import train_test_split
```

In [511…

```
# Train models
gbc = GradientBoostingClassifier(random_state=42).fit(X_train_scaled, y_train)

# Drop use_label_encoder as it's deprecated
xgb = XGBClassifier(eval_metric='logloss', random_state=42)
xgb.fit(X_train_scaled, y_train)

nn_model = MLPClassifier(hidden_layer_sizes=(64, 32),
                         max_iter=200,
                         random_state=42).fit(X_train_scaled, y_train)

# LSTM expects 3D input: [samples, timesteps, features]
n_feats = X_train_scaled.shape[1]
X_train_lstm = X_train_scaled.reshape(-1, 1, n_feats)
X_test_lstm  = X_test_scaled.reshape(-1, 1, n_feats)

lstm_model = Sequential([
    LSTM(32, input_shape=(1, n_feats)),
    Dense(1, activation='sigmoid')
])
lstm_model.compile(optimizer='adam', loss='binary_crossentropy')
lstm_model.fit(X_train_lstm, y_train, epochs=10, batch_size=16, verbose=0)
```

```
C:\Users\Nanoo\anaconda3\Lib\site-packages\sklearn\neural_network\_multilayer_percep
tron.py:690: ConvergenceWarning:

Stochastic Optimizer: Maximum iterations (200) reached and the optimization hasn't c
onverged yet.

C:\Users\Nanoo\anaconda3\Lib\site-packages\keras\src\layers\rnn\rnn.py:199: UserWarn
ing:

Do not pass an `input_shape`/`input_dim` argument to a layer. When using Sequential
models, prefer using an `Input(shape)` object as the first layer in the model instea
d.
```

Out[511…

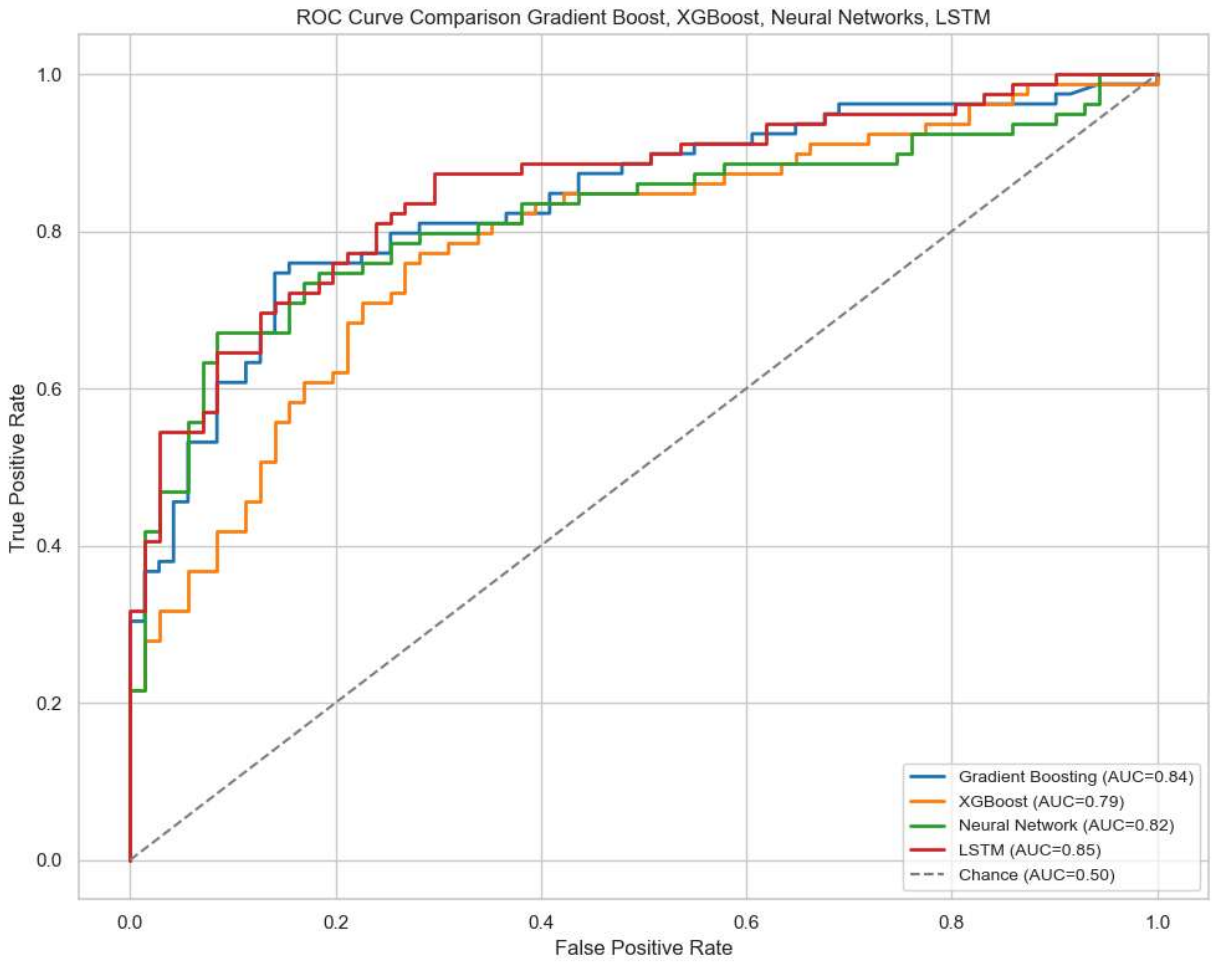
```
<keras.src.callbacks.history.History at 0x250c0399d90>
```

In [521…

```
# Plot ROC curves
sns.set_style("whitegrid")
plt.figure(figsize=(10, 8))
```

```
cmap = plt.get_cmap("tab10")

model_dict = {
    "Gradient Boosting": (gbc,        X_test_scaled),
    "XGBoost":           (xgb,        X_test_scaled),
    "Neural Network":    (nn_model,   X_test_scaled),
    "LSTM":              (lstm_model, X_test_lstm),
}

for idx, (name, (model, Xt)) in enumerate(model_dict.items()):
    if hasattr(model, "predict_proba"):
        y_score = model.predict_proba(Xt)[:, 1]
    elif hasattr(model, "decision_function"):
        y_score = model.decision_function(Xt)
    else:
        y_score = model.predict(Xt).ravel()

    fpr, tpr, _ = roc_curve(y_test, y_score)
    roc_auc = auc(fpr, tpr)

    plt.plot(fpr, tpr,
             color=cmap(idx),
             lw=2,
             label=f"{name} (AUC={roc_auc:.2f})")

# Chance Line
plt.plot([0, 1], [0, 1],
         linestyle="--",
         color="grey",
         label="Chance (AUC=0.50)")

plt.xlabel("False Positive Rate")
plt.ylabel("True Positive Rate")
plt.title("ROC Curve Comparison Gradient Boost, XGBoost, Neural Networks, LSTM")
plt.legend(loc="lower right", fontsize="small")
plt.tight_layout()
plt.show()
```

```
5/5 ━━━━━━━━━━━━━━━━━━━━ 0s 17ms/ste ━━━━━━━━━━━━━━━━━━━━ 0s 6ms/step
```



In [550…
```
conclusion = "- H₁ supported: Biometric measures (especially Oldpeak and MaxHR) out
"- Age and FastingBS also contribute meaningfully once controls are in place."
"- Ensemble methods (Gradient Boosting, XGBoost) and sequence models (LSTM) capture
```

Out[550…
```
'- Ensemble methods (Gradient Boosting, XGBoost) and sequence models (LSTM) captur
e nonlinear interactions and temporal patterns, further boosting discrimination.'
```

In [554…
```
import glob

# grab all PNG, JPG, and SVG files in figures/
eda_figures = sorted(
    glob.glob("figures/*.png")
  + glob.glob("figures/*.jpg")
  + glob.glob("figures/*.svg"))
```

In [603…
```
# ── README GENERATOR ──────────────────────────────────────────────────

import pandas as pd
import subprocess, sys
from datetime import datetime

# ── ▶ Manually defined AUC scores (hard-coded for now)
aucs = {
    "Logistic Regression (Normalized)": 0.86,
    "Logistic Regression (Raw)"       : 0.85,
    "SVM (Normalized)"                : 0.68,
```

```
    "Random Forest (Raw)"             : 0.79,
    "Gradient Boosting"               : 0.84,
    "XGBoost"                         : 0.79,
    "Neural Network"                  : 0.82,
    "LSTM"                            : 0.85}


# ── ▶ earlier in the notebook:
# hypothesis      : str
# df              : pd.DataFrame
# eda_figures     : list[str] (e.g. ["figures/age_dist.png", ...])
# conclusion      : str

# 1) Build Markdown sections
md = []

# Title + timestamp
md.append(f"# Heart Disease Risk Prediction\n\n")
md.append(f"*Generated on {datetime.now().strftime('%Y-%m-%d')}*\n\n")

# Hypothesis
md.append("## Hypothesis\n\n")
md.append(f"{hypothesis}\n\n")

# Data overview
md.append("## Data\n\n")
md.append(f"- Source file: `data/heart.csv`\n")
md.append(f"- Shape: `{df.shape[0]} samples × {df.shape[1]} features`\n\n")

# Summary statistics
md.append("## Summary Statistics\n\n")
summary = df.describe()
md.append(summary.to_markdown() + "\n\n")

# Exploratory Data Analysis
md.append("## Exploratory Data Analysis\n\n")
for fig in eda_figures:
    md.append(f"![EDA]({fig})\n\n")

# Manual summary of EDA techniques
md.append("The following techniques were applied during EDA to uncover patterns and
md.append("- **Histograms** to visualize distributions of key variables\n")
md.append("- **Box Plots** to detect outliers and assess spread\n")
md.append("- **Scatterplots with trendlines** to explore linear relationships\n")
md.append("- **Bubble Plot** with HeartDisease color coding to show multivariate in
md.append("- **Normalized Scatterplots** to compare variables on a common scale\n")
md.append("- **Correlation Analysis** to quantify associations between predictors\n
md.append("- **Outlier Cleaning for Cholesterol** to improve model stability and re


# Modeling results
md.append("## Modeling\n\n")
results_df = pd.DataFrame({
    "Model": list(aucs.keys()),
    "ROC AUC": list(aucs.values())
})
```

```
md.append(results_df.to_markdown(index=False) + "\n\n")

# Conclusions
md.append("## Conclusions\n\n")
md.append(f"{conclusion}\n\n")

# Repository structure
md.append("## Repository Structure\n\n")
md.append("```\n")
md.append("├── data/                # raw and processed datasets\n")
md.append("├── figures/             # EDA & model diagnostic plots\n")
md.append("├── notebooks/           # analysis & experimentation\n")
md.append("│   └── report.ipynb     # this notebook\n")
md.append("├── README.md            # this file (auto-generated)\n")
md.append("├── requirements.txt     # pinned Python dependencies\n")
md.append("└── LICENSE              # license terms\n")
md.append("```\n\n")

# Dependencies via pip freeze
md.append("## Dependencies\n\n")
reqs = subprocess.check_output([sys.executable, "-m", "pip", "freeze"]).decode().sp
for r in reqs:
    md.append(f"- {r}\n")
md.append("\n")

# License
md.append("## License\n\n")
md.append("This project is released under the MIT License. See [LICENSE](LICENSE) f

# 2) Write out README.md
with open("README.md", "w", encoding="utf-8") as f:
    f.writelines(md)

print("✅ README.md generated!")
```

```
✅ README.md generated!
```

In [ ]: